上海创新学院
Shanghai Innovation Institute

An Adversarial Self-Evolving AGENT via Coordinated Examiner–Solver Updating

Authors: Jiaquan Zhang, Zihang Zeng

Dual-Agent Adversarial Loop: Examiner (What-to-Check) × Solver (How-to-Repair)

Examiner
What-to-Check

Adversarial Loop (each repair attempt)

propose test scope / run tests →

Candidate Patch

← generate patch

Solver
How-to-Repair

Controls
- Test selection
- Regression scope
- File inspection strategy
- Validation depth

Learns
- Which tests reveal overfitting
- Which scopes expose hidden failures
- When to escalate: trigger → relevant → full regression

Evaluation Results

Visible tests fail	Visible pass, regression fail	All tests pass (visible + regression)
Examiner catches the error	Examiner wins	Solver wins
records visible failure (−0.5)	records regression outcome (−1.0)	both sides record success (+1.5)
Solver learns this strategy does not fit these features	Solver learns the patch style overfits	success strategy becomes reusable
	retry-after-feedback remains a useful skill (+0.75)	test-scope choice is reinforced

Controls
- Patch style selection
- Repair skill choice
- Failure reflection
- Success strategy reuse

Learns
- Which patch patterns succeed on similar bugs
- Which strategies are dead ends
- What prior experience can be reused

Iterative Memory Updates (seven memory stores across both agents)

Examiner Memory
(What-to-Check dimension)

test_selection
updated every outcome; records which test scope is most informative (−1.5 solved / −1.0 overfit caught)

test_skill_memory
updated every outcome; records which testing skills work (e.g., regression-relevant-caught-overfit: +0.75)

regression_outcomes
updated on regression checks; stores failed style / scope pairs and creates future warnings

Attempt outcome → Update both memories

- scores
- skills
- strategies / warnings

Solver Memory
(How-to-Repair dimension)

patch_ranking
updated every outcome; which patch styles succeed (+1.5) / −0.75 compile fail / −0.5 visible fail

repair_skill_memory
updated every outcome; which repair skills transfer (e.g., retry-after-feedback)

failure_reflections
updated on failure; transferable lessons with case-specific details removed

success_strategies
updated on success; archived reusable winning strategies

Cross-Task Transfer: Reusing Feature-Keyed Knowledge

Features	Keys
project:Lang	cla:ssnumberutils
exception:assertionerror	trigger:testPrecision

Memory is keyed by features, not case IDs, enabling semantically meaningful transfer.

Bug A fixed	Solver: numeric-conversion style succeeds Examiner: relevant scope catches overfit
Bug B arrives (similar features)	Solver: inherits patch guidance Examiner: inherits preferred scope + regression warnings
Direct benefit: fewer attempts higher success rate	Transfer reduces search cost and improves repair success rate
30+ bug Defects4J experiment: memory features expanded from 5 to 109	Feature memory grows across tasks from 5 to 109 during the 30-bug run

Why the Adversarial Design Is Necessary

Deterministic ablation: 4 bugs

Configuration	Solved	Result
No memory (feedback only)	0/4	Random repair choices; fixed checking scope; overfit- and duplicate-risk
Examiner memory only	0/4	Correct checking, but weak repair guidance
Solver memory only	1/4	Correct patch style, but weak regression protection
Both memories (full adversarial)	4/4	Overfit prevented, duplicates avoided, complementary coverage

Why Is It Self-Evolving?

Traditional RL/ML	This System
updates reward weights	updates external memory tables
requires gradient computation / retraining	requires only test execution
learns within a single task	learns across tasks via feature transfer
checker reward is fixed	Examiner evolves & testing strategy
only the solver learns	both Examiner and Solver learn
forgetting can be catastrophic	forgetting is controlled with decay + don't-go cap
improvement requires retraining	improvement is immediate on next run; big benefit

Final Defects4J Benchmark Results

Metric	Self-Evolving Agent	Baseline
Pass@1	22/30	0/30
Pass@3	26/30	0/30

Adversarial Drive • Memory Transfer • Self-Evolution • Stronger with Every Bug